# The Von Neumann Model

CSE 100/1000/1100: Introduction to Computer Engineering

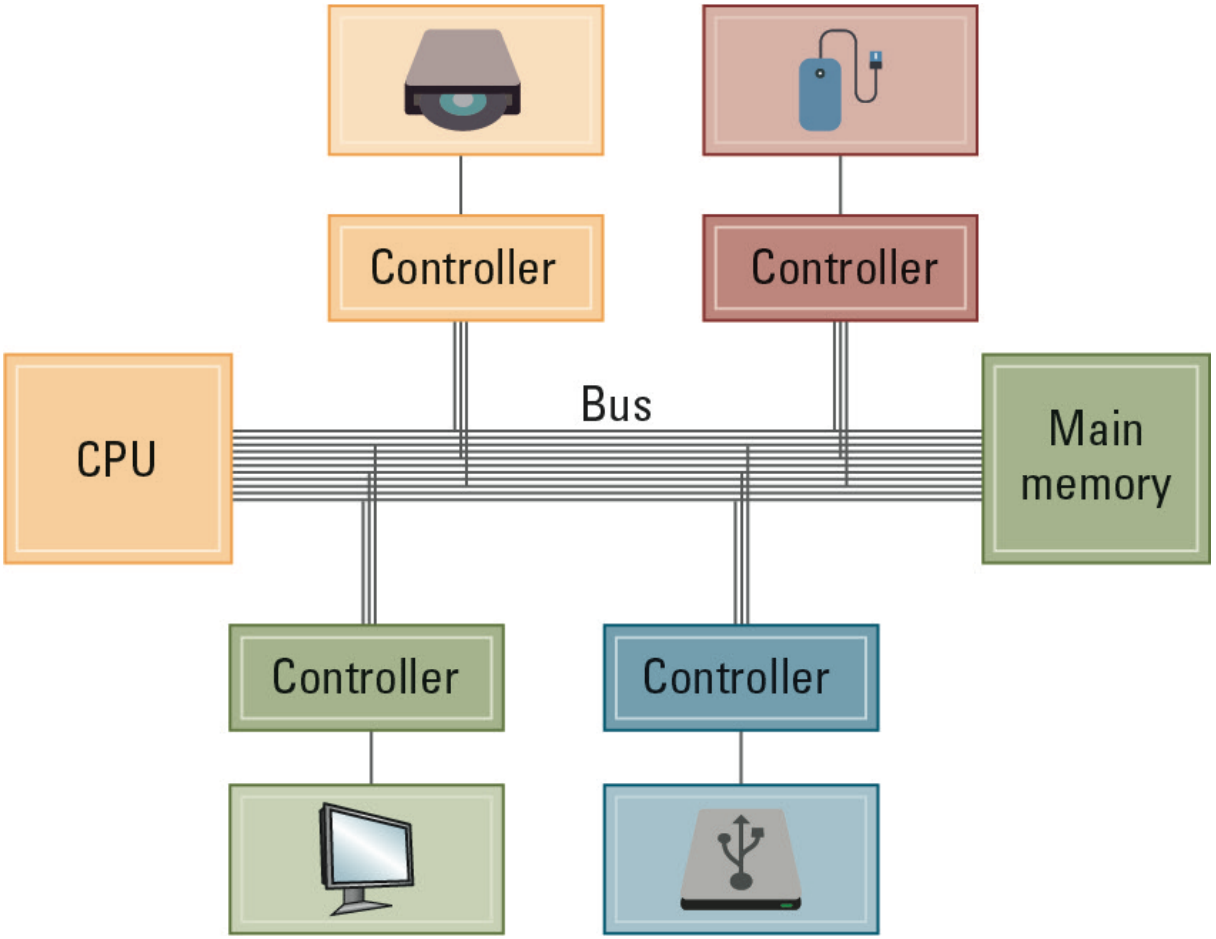
**Instructor:**
Fatma CORUT ERGİN

*Slides adapted from*

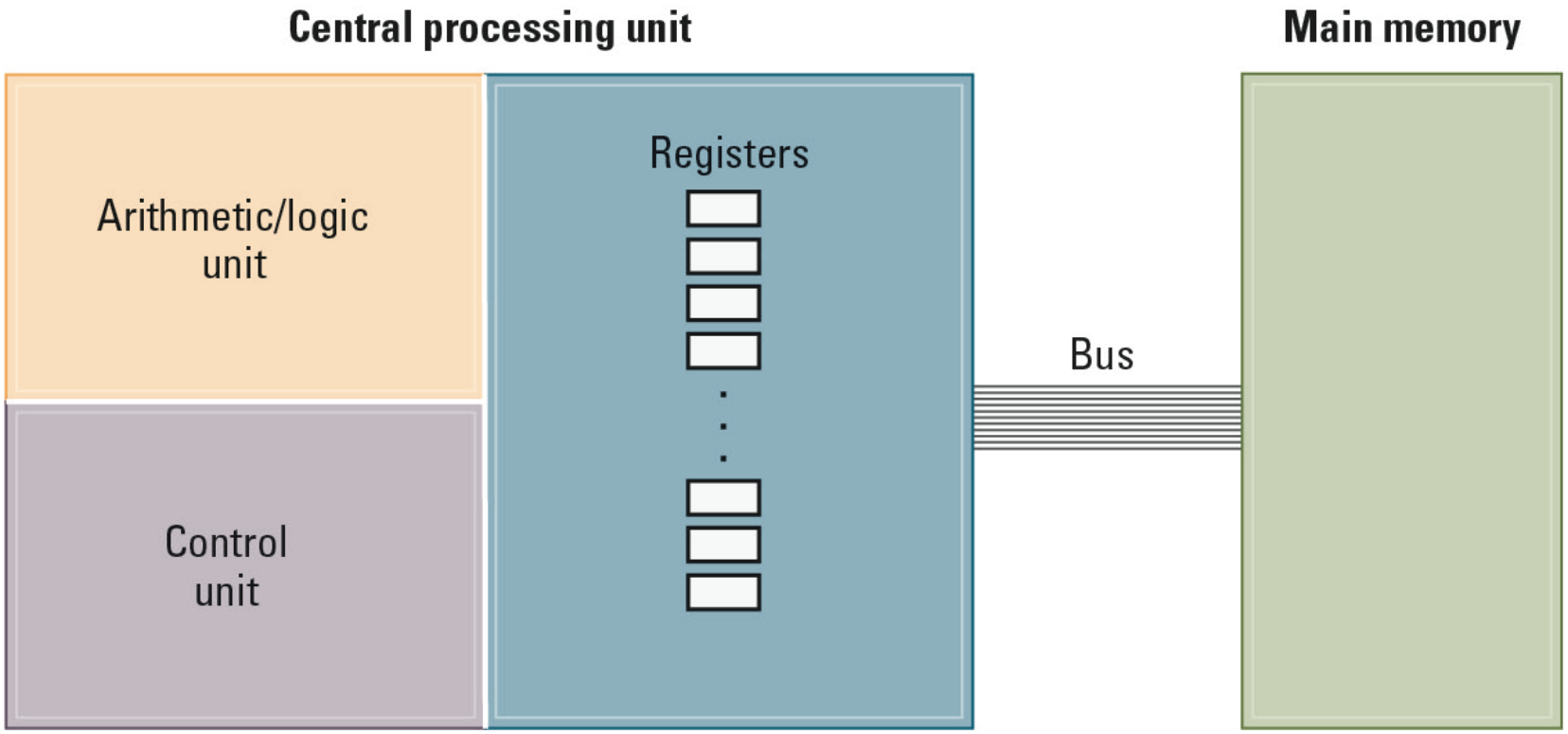
- *McGraw Hill Companies, and*
- *Walid A. Najjar & Brian J. Linard*

*slides*

# Controllers attached to a machine's bus



# CPU and main memory connected via a bus



# Memory

## Each location has an *address* and *contents*

- ***Address:*** **bit pattern that uniquely identifies a memory location**
- ***Contents:*** **bit pattern stored at a given address.**

## Address Space:

- **The total number of memory locations available.**
- **e.g. a 28 bit address provides an address space of $2^{28}$ locations.**

➢ *The LC-3 has an address space of $2^{16}$ locations*

➢ *i.e. it uses 16 bit addresses*.

**Main memory**

| Address | Cells |
|---|---|
| 0xA0 | 0x15 |
| 0xA1 | 0x6C |
| 0xA2 | 0x16 |
| 0xA3 | 0x6D |
| 0xA4 | 0x50 |
| 0xA5 | 0x56 |
| 0xA6 | 0x30 |
| 0xA7 | 0x6E |
| 0xA8 | 0xC0 |
| 0xA9 | 0x00 |

# Memory

**$2^k$ x $m$ array of stored bits**

**Address**

- **unique (*k*-bit) identifier of location**
- **address space: $2^k$**

**Contents**

- ***m*-bit value stored in location**

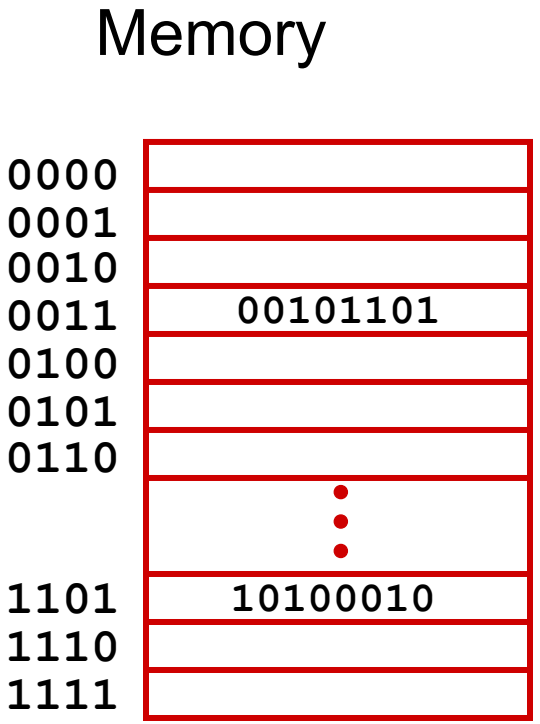
**Basic Operations:**

**LOAD**

- **read a value from a memory location**

**STORE**

- **write a value to a memory location**

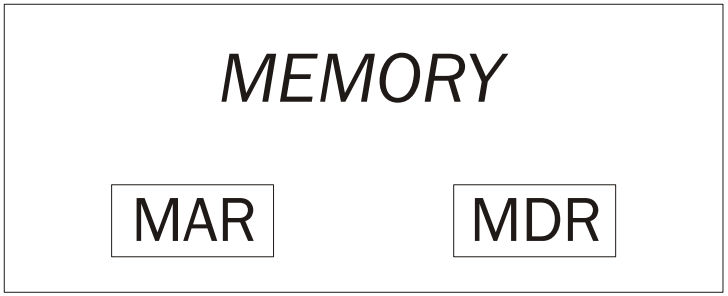


# Interface to Memory

**How does processing unit get data to/from memory?**

**MAR: Memory Address Register**

**MDR: Memory Data Register**




**To LOAD from a location (A):**

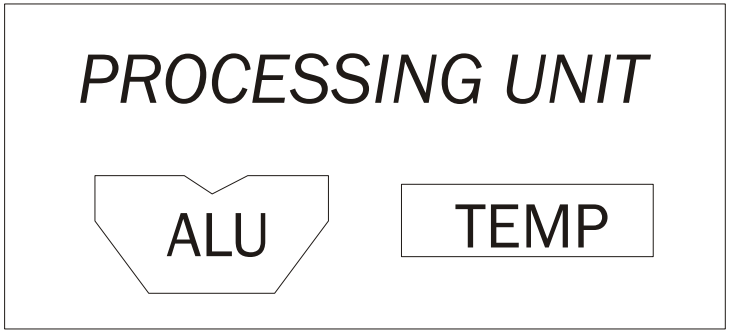
1. **Write the address (A) into the MAR.**
2. **Send a "read" signal to the memory.**
3. **Write the data into MDR.**

**To STORE a value (X) to a location (A):**

1. **Write the address (A) into the MAR.**
2. **Write the value (X) to the MDR.**
3. **Send a "write" signal to the memory.**
4. **The data in MDR will be written into the memory location whose address is in MAR.**

# Central Processing Unit

- **Does the actual work!**
- **number of bits normally processed by ALU in one instruction is called Word Size**
  - **also width of registers**
  - ➢ *LC-3 word size is 16 bits*




## Arithmetic and Logic Unit (ALU)

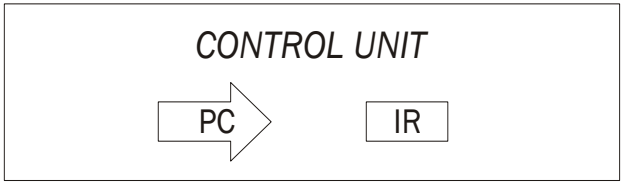
- **Performs basic operations: *add, subtract, and, not*, etc.**
  - ➢ *LC-3 performs ADD, AND, NOT*

## Registers (TEMP)

- **Small, temporary storage**
- **Operands and results of functional units**
- **Close to the ALU (much faster access than RAM)**
  - ➢ *LC-3 has eight registers (R0, …, R7), each 16 bits wide*

# Control Unit

**Coordinates all actions needed to execute the instruction**



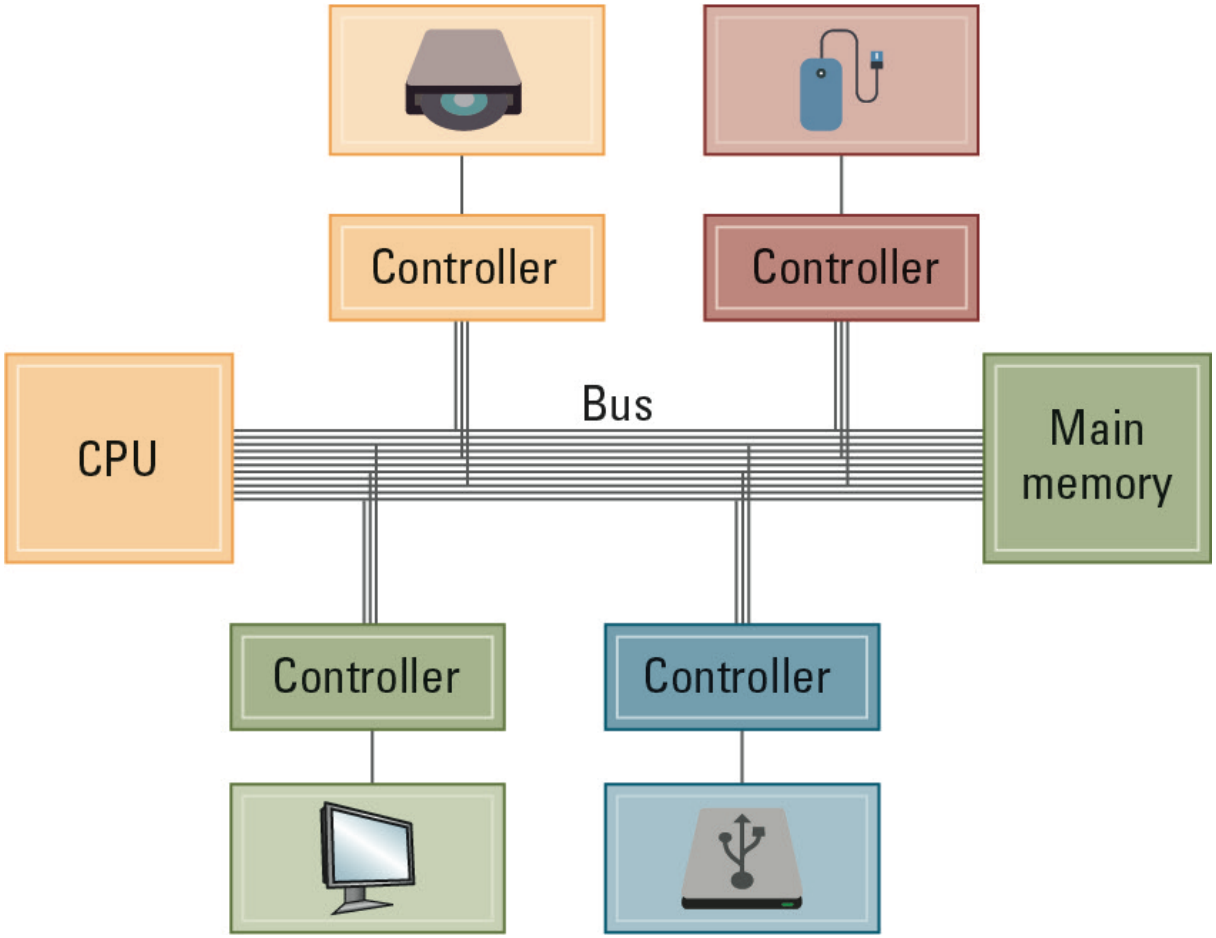

## Control Unit

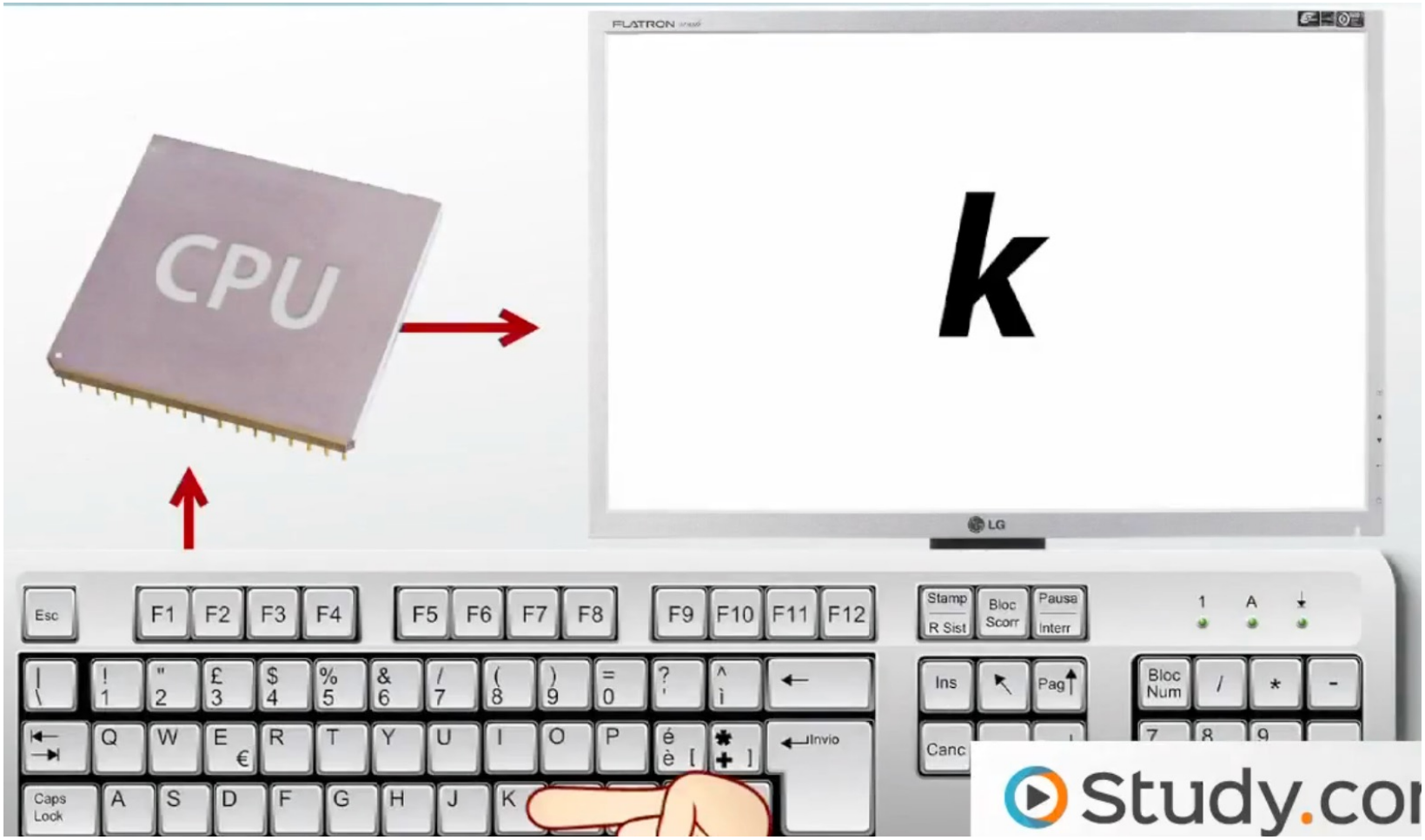
- communicates with memory via the PC and IR
- fetches instruction from memory
  - the instruction's address is stored in the PC
- decodes the instruction, generating signals that tell the other components what to do

**Instruction Register (IR) contains the *current instruction*.**

**Program Counter (PC) contains the *address of the next instruction* to be executed (aka *Instruction Pointer*).**

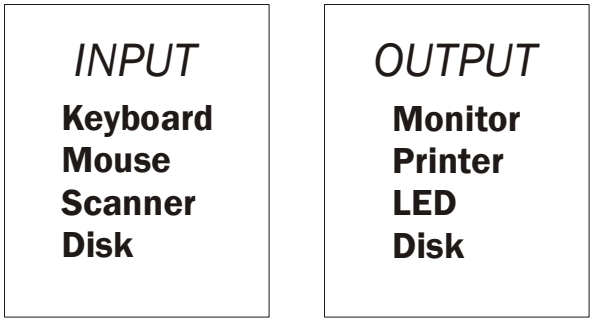
# Controllers attached to a machine's bus




CPU
k
FLATRON
LG
Esc
F1
F2
F3
F4
F5
F6
F7
F8
F9
F10
F11
F12
Stamp
R Sist
Bloc
Scorr
Pausa
Interr
Ins
Pag
Bloc
Num
Canc
Invio
Caps
Lock
Study.com

# Input and Output

**Devices for getting data into and out of computer memory**




**Each device has its own interface, usually a set of registers (like the memory's MAR and MDR)**

- *LC-3 supports keyboard (input) and monitor (output)*

**Some devices provide both input and output**

- **disk, network**

**Program that controls access to a device is usually called a *driver*.**

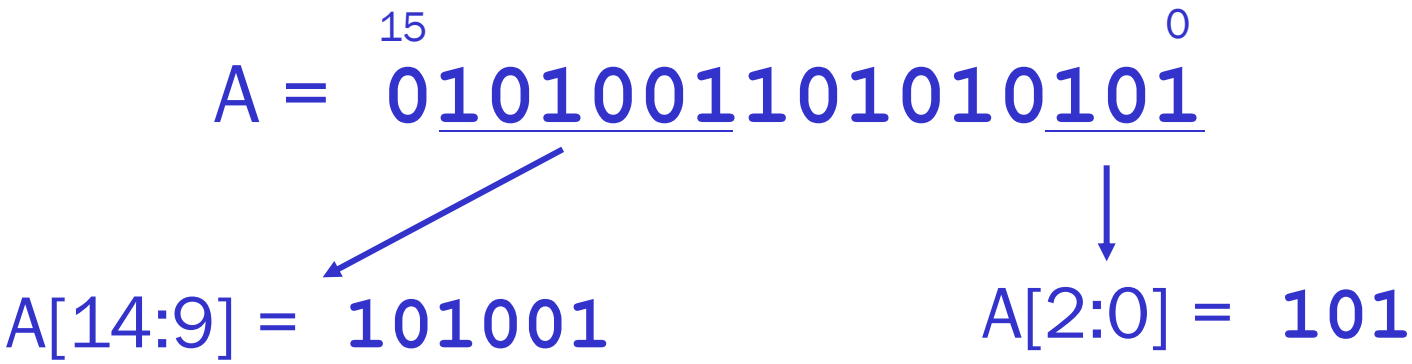
# Representing Multi-bit Values

**Number bits from right (0) to left (n-1)**

- **just a convention -- could be left to right, but must be *consistent***

**Use brackets to denote range:**

D[*ℓ:r*] **denotes bit *ℓ* to bit *r*, from *left* to *right***



# Notations

## Sets of Bits

- A[3:0] denotes a set of 4 bits: $A_3$, $A_2$, $A_1$, $A_0$
- The content of an n-bit register R is referred to as R[n-1:0]
  - $R_{n-1}$ is the most significant bit (MSB), or leftmost bit
  - $R_0$ is the least significant bit (LSB), or rightmost bit
  - Given R[31:0] → R[7:4] refers to the four bits from $R_7$ to $R_4$

## Bit Assignment

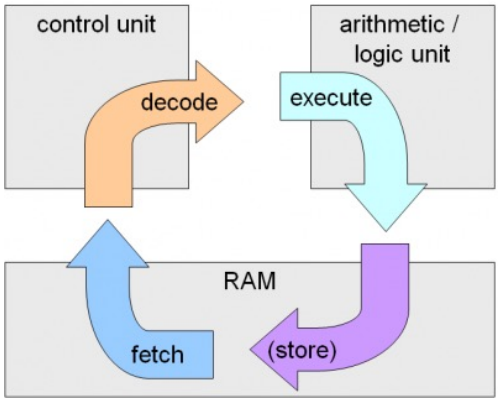
- R[5:0] ⇐ I[13:8]
  - Means that bits 5 to 0 of register R get assigned the values of bits 13 to 8 of register I.

## Contents

- (Reg1) means "content of Reg1"
- Mem[*loc*] means "content of memory location *loc*"

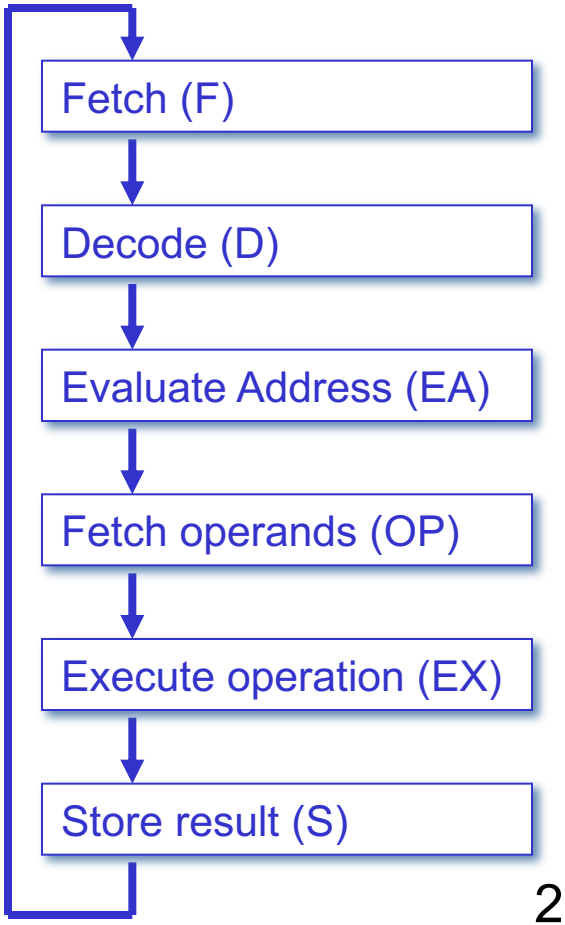
# Instruction Cycle

The Control Unit orchestrates the complete execution of each instruction




## Six phases of the complete Instruction Cycle

- **Fetch**: load IR with instruction from memory
- **Decode**: determine action to take
  (set up inputs for ALU, RAM, etc.)
- **Evaluate address**: compute
  memory address of operands, if any
- **Fetch operands**: read operands from memory
  or registers
- **Execute**: carry out instruction
- **Store results**: write result to destination
  (register or memory)



# Machine Instructions

**The instruction is the fundamental unit of work.**

**An instruction is encoded as a sequence of bits.**
***(Just like data!)***

- Often, but not always, instructions have a fixed length, such as 16 or 32 bits.
- Control unit interprets instruction:

**Specifies two things:**

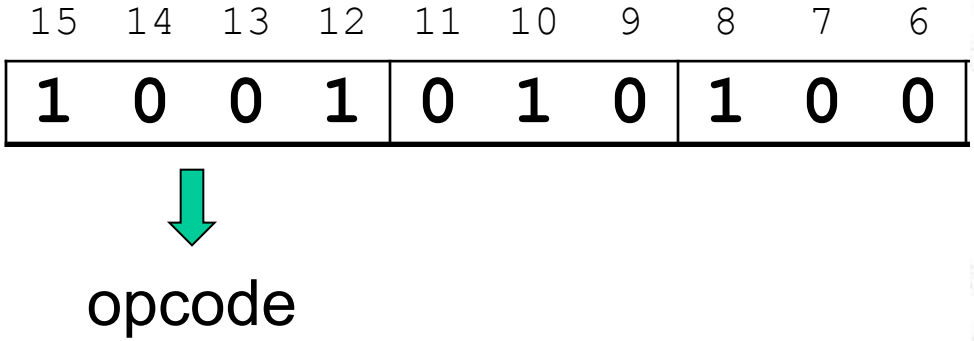
- ***opcode*: operation to be performed**
- ***operands*: data/locations to be used for operation**

**A computer's instructions and their formats is known as its** *Instruction Set Architecture (ISA)***.**

# LC-3 Instructions

- *LC-3 has 15 different instructions in*
- *LC-3 has 16-bit instructions*
- *LC-3 has 4-bit opcode → bits [15:12]*

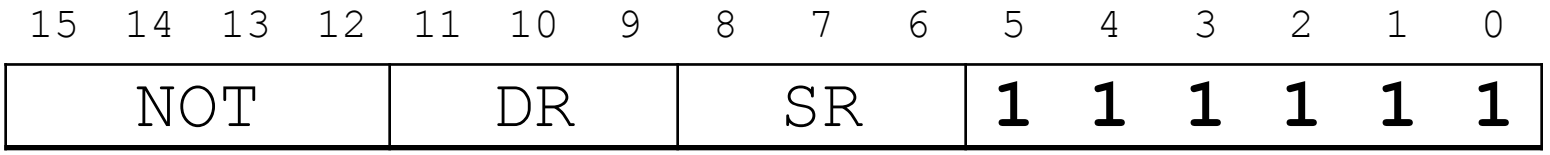
## Example LC-3 Instruction




| | 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ADD+ | 0001 | | | | DR | | | SR1 | | | 0 | 00 | | SR2 | |
| ADD+ | 0001 | | | | DR | | | SR1 | | | 1 | imm5 | | | |
| AND+ | 0101 | | | | DR | | | SR1 | | | 0 | 00 | | SR2 | |
| AND+ | 0101 | | | | DR | | | SR1 | | | 1 | imm5 | | | |
| BR | 0000 | | | | n | z | p | PCoffset9 | | | | | | | |
| JMP | 1100 | | | | 000 | | | BaseR | | | 000000 | | | | |
| JSR | 0100 | | | | 1 | PCoffset11 | | | | | | | | | |
| JSRR | 0100 | | | | 0 | 00 | | BaseR | | | 000000 | | | | |
| LD+ | 0010 | | | | DR | | | PCoffset9 | | | | | | | |
| LDI+ | 1010 | | | | DR | | | I | PCoffset9 | | | | | | |
| LDR+ | 0110 | | | | DR | | | BaseR | | | offset6 | | | | |
| LEA+ | 1110 | | | | DR | | | PCoffset9 | | | | | | | |
| NOT+ | 1001 | | | | DR | | | SR | | | 111111 | | | | |
| RET | 1100 | | | | 000 | | | 111 | | | 000000 | | | | |
| RTI | 1000 | | | | 000000000000 | | | | | | | | | | |
| ST | 0011 | | | | SR | | | PCoffset9 | | | | | | | |
| STI | 1011 | | | | SR | | | PCoffset9 | | | | | | | |
| STR | 0111 | | | | SR | | | BaseR | | | offset6 | | | | |
| TRAP | 1111 | | | | 0000 | | | trapvect8 | | | | | | | |

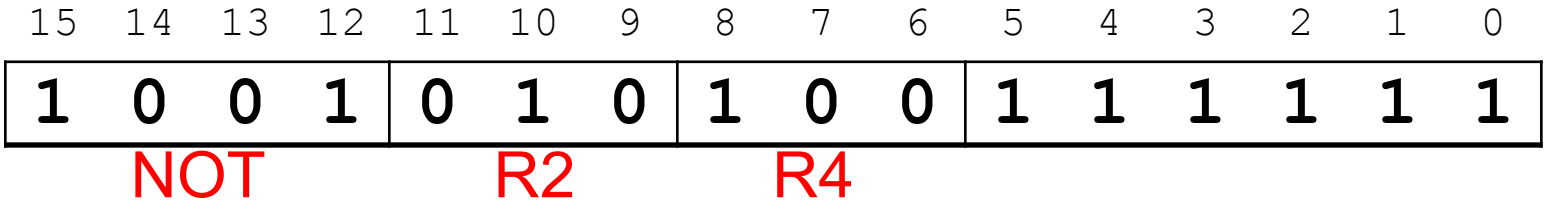
# Example: LC-3 NOT Instruction

**Source and destination of NOT are registers.**




***General Usage:*** **NOT DR, SR**

*"Take the bitwise complement (NOT) of the contents of SR, and store the result in DR."*



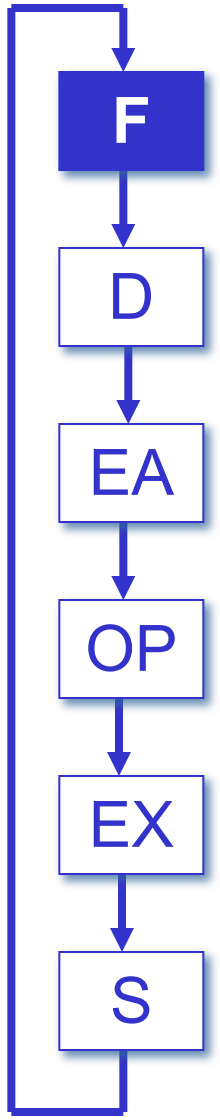

***Example Instruction:*** **NOT R2, R4**

**=> R2 = NOT(R4)**

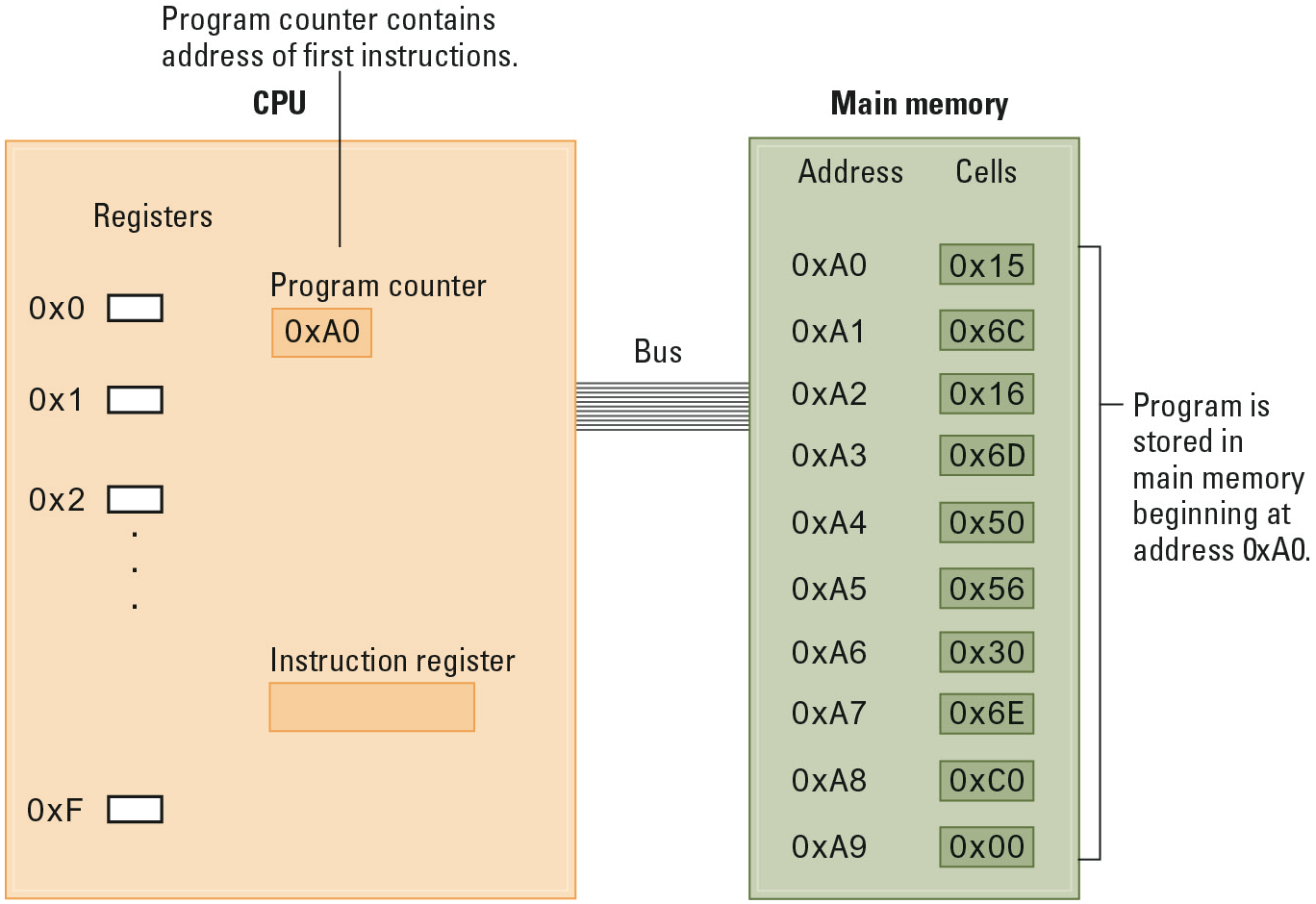
# Instruction Cycle: FETCH

**Load next instruction (at address stored in PC) from memory into Instruction Register (IR).**

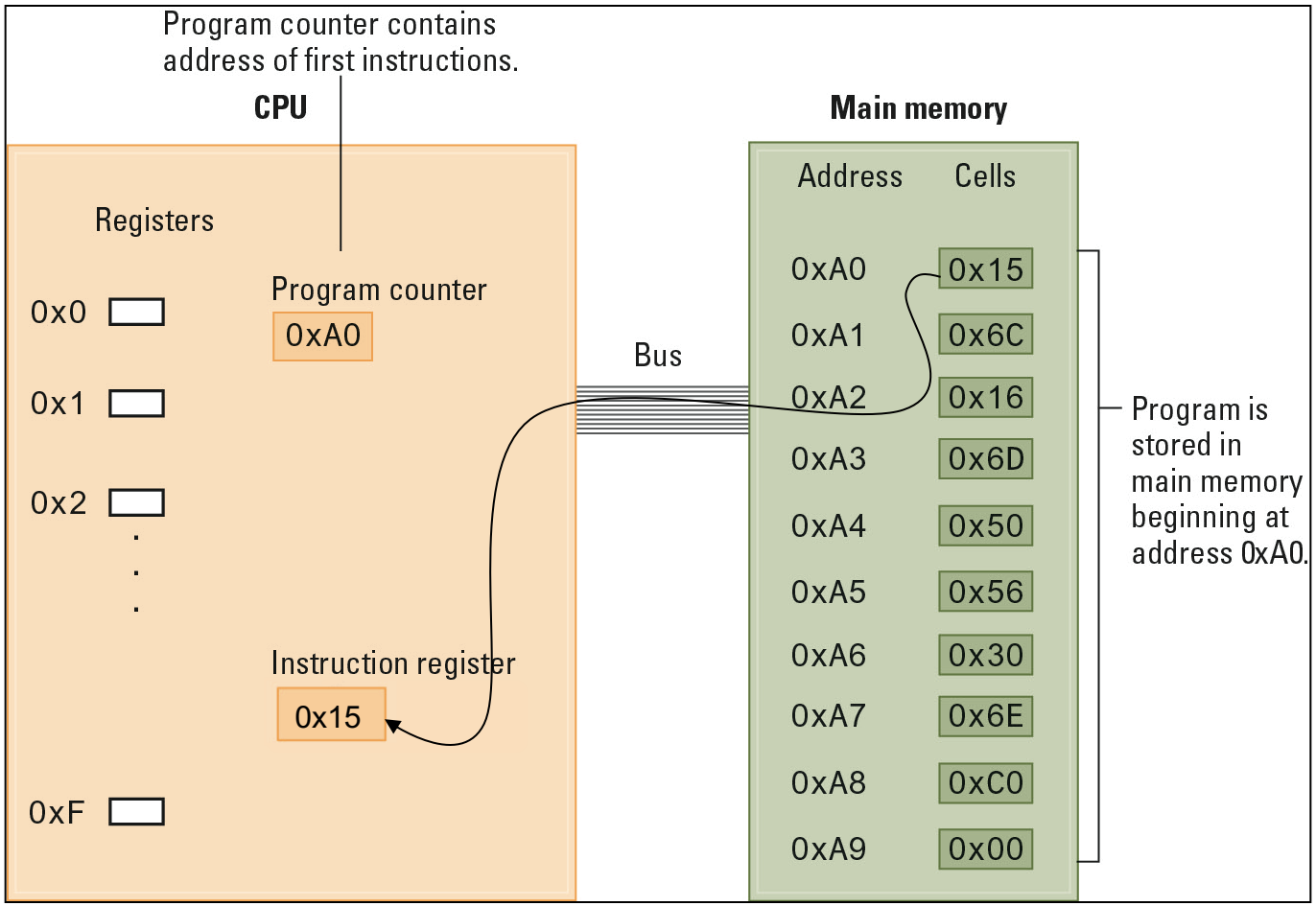
- (MAR) ← (PC) ; copy value of PC into MAR
- (PC) ← (PC) + 1 ; increment the value of PC
- (MDR) ← Mem[MAR] ; read memory location to MDR
- (IR) ← (MDR) ; copy contents of MDR into IR



# A program stored in main memory ready for execution

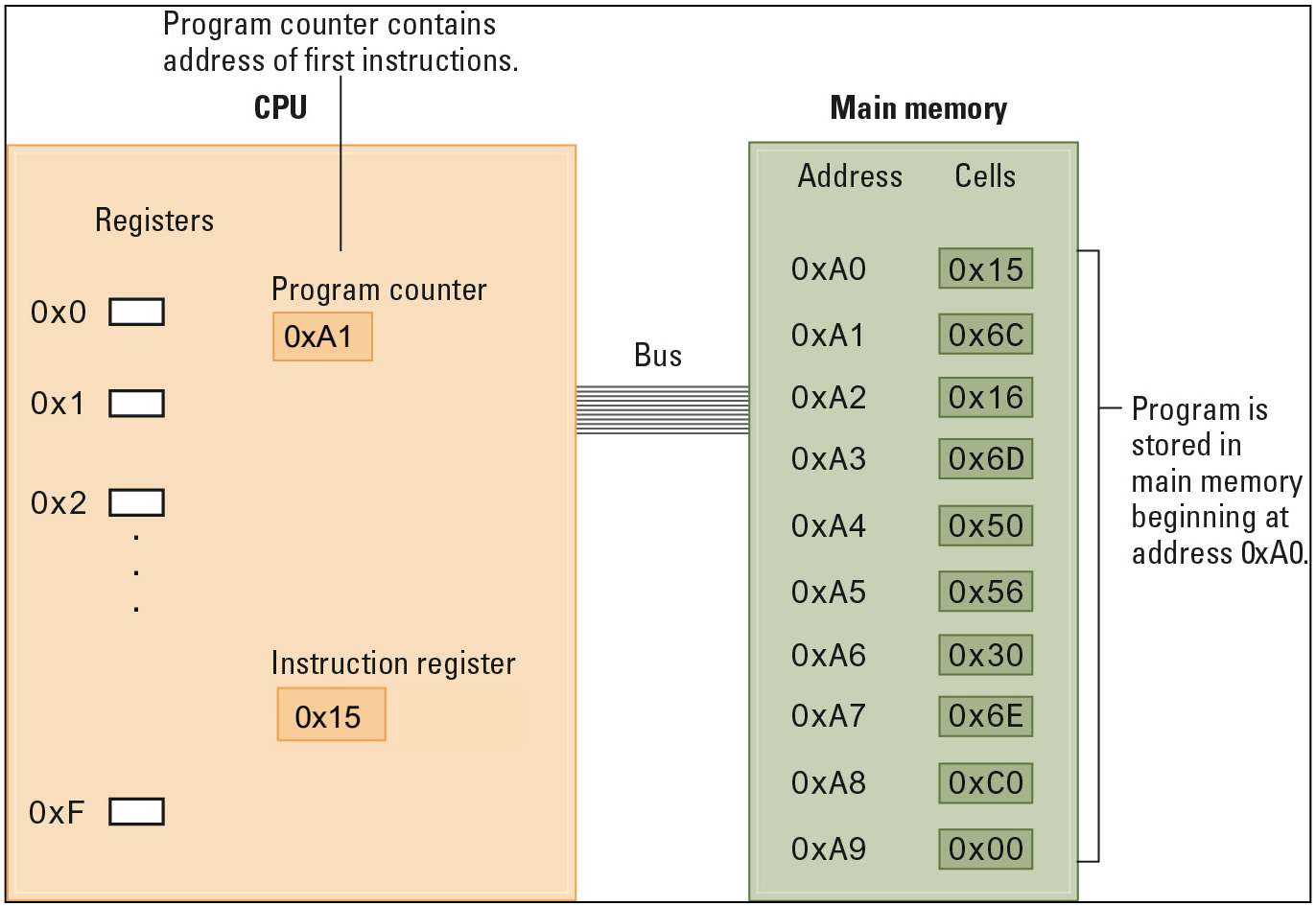


# Performing the fetch step of the machine cycle




**a.** At the beginning of the fetch step, the instruction starting at address 0xA0 is retrieved from memory and placed in the instruction register.

# Performing the fetch step of the machine cycle




**b.** Then the program counter is incremented so that it points to the next instruction.

# Instruction Cycle: FETCH

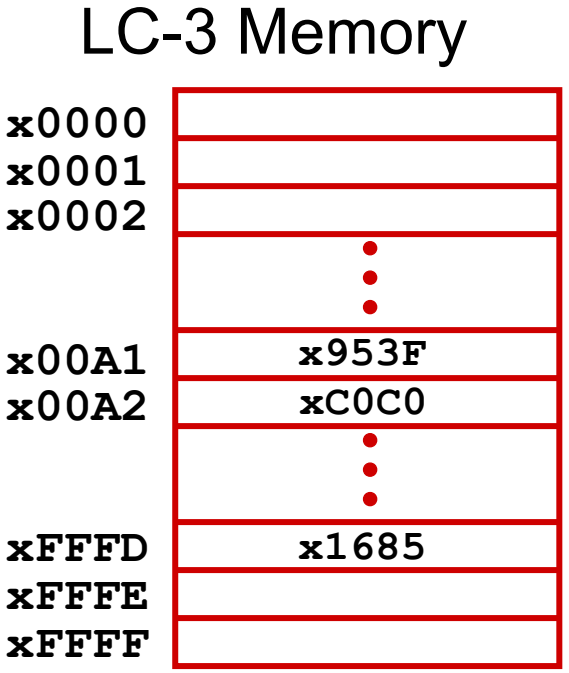
- LC-3 example:
  - Assume PC= x00A1
  - After FETCH phase:

    MAR = x00A1,

    PC = x00A2,

    MDR = x953F,

    IR = x953F

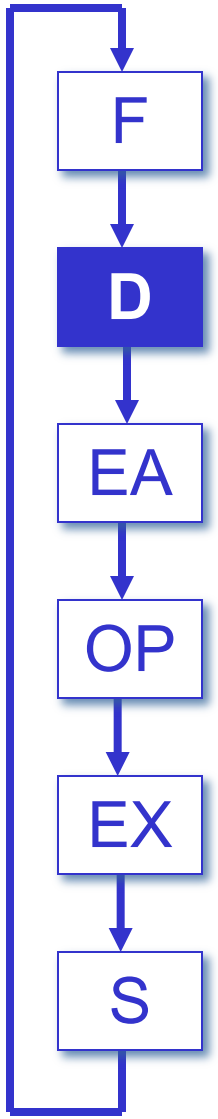


# Instruction Cycle: DECODE

**First, identify the opcode.**

- **The opcode is input to a decoder, which sets up the following sequence of events according to the instruction.**

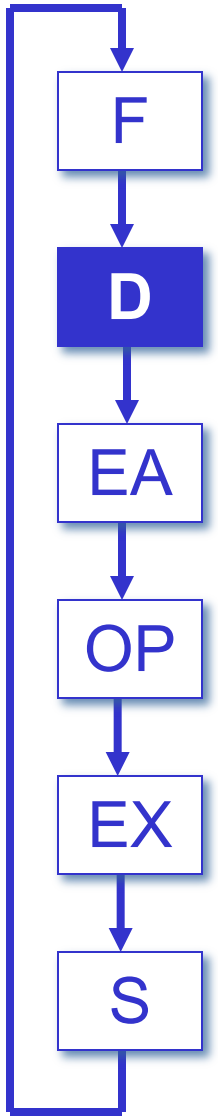
**Depending on opcode, identify other operands from the remaining bits.**



# Instruction Cycle: DECODE

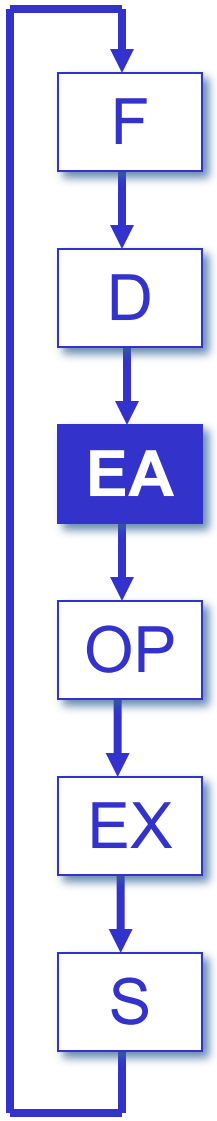
- In LC-3:
  - opcode is always the first four bits of instruction
  - 4-to-16 decoder is used

- LC-3 example:
  - Assume IR = x953F (1001 0101 0011 1111)
    - IR[15:12] is opcode → 1001 means a NOT instruction
    - IR[11:9] is destination register → R2
    - IR[8:6] is source register → R4



# Instruction Cycle: EVALUATE ADDRESS

**For instructions that require memory access, compute address used for access.**




- LC-3 NOT instruction does not require the Evaluate Address phase

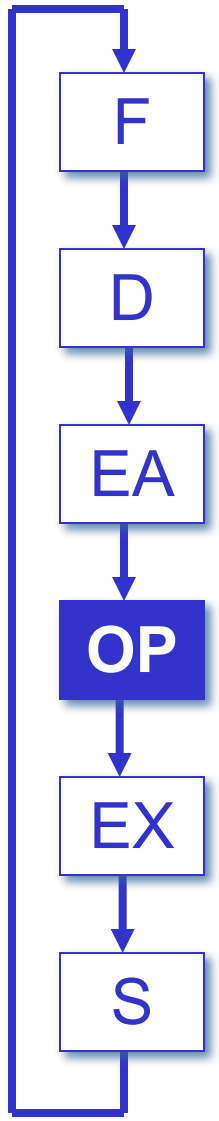
- LC-3 LDR instruction requires the Evaluate Address phase

# Instruction Cycle: FETCH OPERANDS

**Obtain source operands needed to perform operation.**

**Examples:**

- **load data from memory**
- **read data from register file**

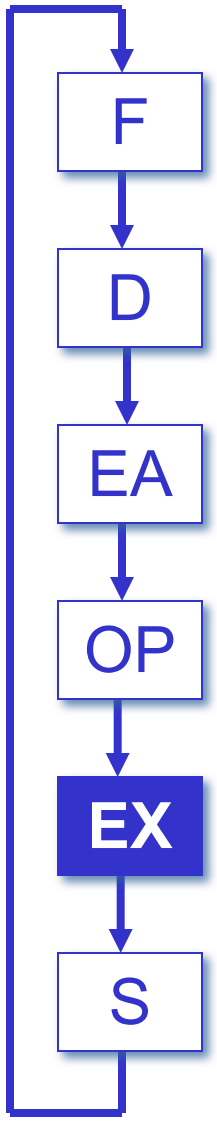


# Instruction Cycle: EXECUTE

**Perform the operation,**
**using the source operands.**

**Examples:**

- **send operands to ALU and assert ADD signal**
- **do nothing (e.g., for loads and stores)**

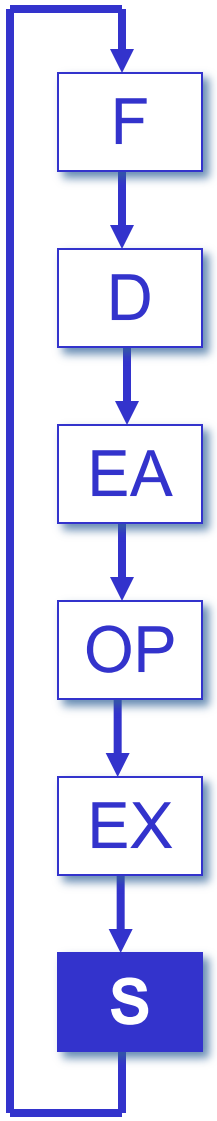


# Instruction Cycle: STORE RESULT

**Write results to destination.**
**(register or memory)**

**Examples:**

- **result of NOT is placed in destination register**
- **result of memory load is placed in destination register**
- **for store instruction, data is stored to memory**
  - ➢ **write address to MAR, data to MDR**
  - ➢ **assert WRITE signal to memory**



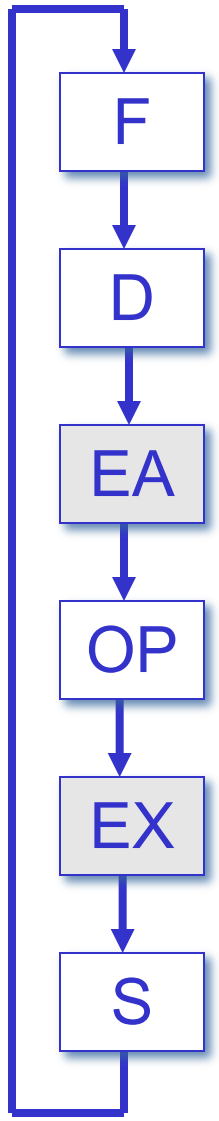
# Instruction Cycle

## Start over …

- The control unit just keeps repeating this whole process.
- So, it now Fetches a new instruction from the address currently stored in the PC.

## Note: Some instructions don't need all 6 phases

- If only using registers, skip Evaluate Address
- If only moving data, skip Execute



# Changing the Sequence of Instructions

**In the FETCH phase,
we increment the Program Counter (PC) by 1.**

**What if we don't always want to execute the instruction that follows this one?**

- **examples: loop, if-then, function call**

**Control instructions set the PC to a new value during the Execute phase, so the next instruction comes from a different place in the program**

# Types of Instruction

## *Computational* Instructions

- **process data (addition, logical operations, etc.)**
- +, -, *, /, AND, OR, SHIFT, ROTATE

## *Data Movement* Instructions

- **move data between memory locations and registers.**
- LOAD, STORE

## *Control* Instructions

- **change the sequence of execution of instructions in the program**
- JUMP, BRANCH

# Types of Instruction

## *Computational* Instructions

- **process data (addition, logical operations, etc.)**
  - ➢ *in LC-3: ADD, AND, NOT instructions*

## *Data Movement* Instructions

- **move data between memory locations and registers.**
- *in LC-3: LD, LDI, LDR, LEA, ST, STI, STR instructions*

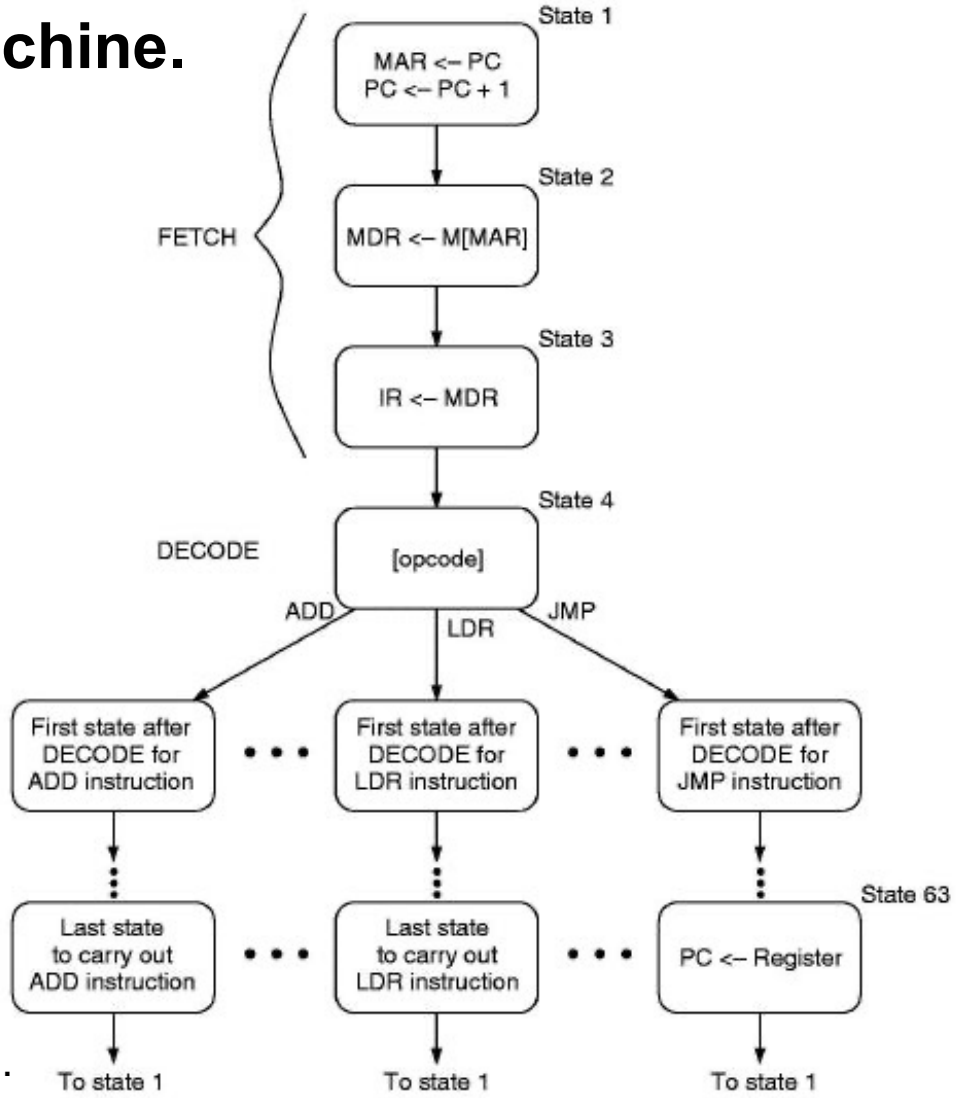
## *Control* Instructions

- **change the sequence of execution of instructions in the program**
- *in LC-3: BR, JSR/JSRR, JMP, RTI, TRAP instructions*

# Control Unit State Diagram

**The control unit is a state machine.**

**Here is part of a simplified state diagram for the LC-3:**




A more complete state diagram is in Appendix C. It will be more understandable after Chapter 5.

# Stopping the Computer

**Control unit will repeat instruction processing sequence as long as computer is running.**

- **If not processing instructions from your application, then it is processing instructions from the Operating System (OS).**
- **The OS is a special program that manages processor and other resources.**
- **We can finish processing completely by stopping the machine cycle - i.e. by stopping the clock.**